

Roteiro do vídeo: bcrypt (slides + código + teste no site)

Duração: cerca de 6 a 7 minutos. Isabela: apresenta todos os slides (cerca de 3:30). Paulo Victor: mostra o código, roda e explica o resultado, e faz o teste no site gerador de bcrypt (cerca de 3:30). Os valores de sal e hash mudam a cada execução; leiam o que aparecer na tela.

Antes de gravar: (1) slides abertos em tela cheia; (2) terminal na pasta do projeto e o arquivo `bcrypt_seminario.py` aberto em um editor; (3) o site gerador de bcrypt aberto em uma aba; (4) rodar uma vez para testar (leva uns 15 s até o menu).

PARTE I - Slides (Isabela, ~3:30)

Slide 1 - Capa (0:15)

Olá, somos a Isabela e o Paulo Victor, tema 9: bcrypt. Vamos explicar como funciona, mostrar o nosso código em Python puro, sem nenhum import, e testar o resultado em um site.

Slide 2 - O que é o bcrypt (0:30)

O bcrypt é uma função de hash feita para guardar senhas, criada em 1999 por Provos e Mazières e baseada na cifra Blowfish. Três ideias: hash de mão única, ou seja, do hash não se volta à senha; sal aleatório de 16 bytes, que faz a mesma senha gerar hashes diferentes; e custo ajustável, em que o trabalho se repete 2 elevado ao custo vezes. Embaixo vemos o hash de 60 caracteres: versão, custo, sal de 22 caracteres e hash de 31.

Slide 3 - Os 6 passos (0:50)

O algoritmo tem seis passos. Um: a senha vira bytes, ganha um byte nulo e é cortada em 72 bytes. Dois: o estado inicial vem dos dígitos de pi. Três: mistura-se o sal e a senha, com 521 cifragens Blowfish. Quatro: repete-se 2 elevado ao custo vezes a mistura da senha e do sal; é aqui que mora a lentidão. Cinco: cifra-se 64 vezes um texto fixo. Seis: monta-se o hash final. Em cada caixa aparece o nome da função no código.

Slide 4 - Fluxo completo (0:25)

Aqui o mesmo fluxo em diagrama. O banco guarda só o hash. O sal e o custo ficam dentro dele, e é por isso que o login consegue refazer o cálculo. Com custo 5 são 34.057 cifragens.

Slide 5 - Exemplo prático (0:35)

Um exemplo real com a senha 'senhaeu' e custo 5. Vemos o primeiro valor do vetor P mudando a cada etapa e o hash final. No login, o sistema lê custo e sal do hash, refaz o cálculo com a senha digitada e compara em tempo constante. Trocar uma letra muda o hash por completo.

Slide 6 - Por que é seguro (0:30)

O bcrypt não impede o palpite: ele torna cada palpite caro. O sal único derrota as rainbow tables, o custo acompanha o hardware, as etapas são em sequência e a comparação é em tempo constante, o que evita ataque de temporização. Os números de tentativas por segundo vêm do artigo citado no slide.

Slide 7 - Vantagens e limitações (0:30)

As vantagens: feito para senhas, sal embutido, custo ajustável e suportado em todas as linguagens. As limitações: só 72 bytes de senha contam, usa apenas 4 KB de memória, o

byte nulo é proibido e senha fraca continua fraca.

Slide 8 - bcrypt x scrypt x Argon2id (0:20)

Comparando com o scrypt e o Argon2id: os dois exigem memória configurável e resistem melhor a GPUs. A OWASP recomenda o Argon2id para sistemas novos, mas o bcrypt continua aceito.

Slide 9 - Referências (0:10)

Estas são as referências. Agora o Paulo vai mostrar o código funcionando.

PARTE II - Código (Paulo, ~1:00)

NA TELA: editor com `bcrypt_seminario.py`; rolar devagar pelos blocos

Este é o nosso código, em um único arquivo e sem nenhum import. Ele tem cinco blocos. A: calcula os dígitos de pi com a fórmula de Machin. B: implementa a cifra Blowfish, com a função F, as 16 rodadas e a função `expandir_chave`, que é a operação central. C: o base64 próprio do bcrypt. D: o sal aleatório, com um gerador simples, só para demonstração. E: junta tudo em `hash_bruto`, `gerar_hash` e `verificar_senha`.

NA TELA: parar em `hash_bruto`, no laço `for _ in range(2 ** custo)`

Aqui está a lentidão proposital: o laço repete 2 elevado ao custo vezes duas chamadas de `expandir_chave`. Cada +1 de custo dobra o tempo.

PARTE III - Rodar e explicar o resultado (Paulo, ~1:30)

NA TELA: terminal: `python bcrypt_seminario.py`

Vou rodar. Parte 1: o programa confere que os dígitos de pi que calculamos são iguais aos do artigo original do Blowfish, `P[0]` igual a 243F6A88, e mostra três rodadas de Feistel. Parte 2: o bcrypt passo a passo com a senha 'abc' e custo 4. Vemos o `P[0]` mudando a cada etapa, o hash final e a contagem de 17.385 cifragens, igual à fórmula. Parte 3: três vetores do bcrypt oficial; o nosso código gera exatamente o mesmo hash e aparece OK nos três.

Parte 4: a mesma senha gera dois hashes diferentes por causa do sal; o login certo dá True e o errado dá False; e o ataque de dicionário acha a senha fraca na sexta tentativa.

Parte 5: duas senhas que só diferem depois do byte 72 geram o mesmo hash.

PARTE IV - Teste no site (Paulo, ~1:00)

NA TELA: menu: digitar 1, senha `videoteste`, custo 4; copiar o hash `$2b$04$...`

Agora a prova externa. Vou gerar um hash da senha 'videoteste' com o nosso programa e copiar o resultado.

NA TELA: no site gerador de bcrypt: aba de verificação (Check/Compare); colar senha e hash

Neste site gerador de bcrypt, colo a senha e o hash. O site informa que bate. Ou seja, um bcrypt feito por terceiros reconhece o hash do nosso código como válido.

NA TELA: no site: gerar hash de 'videoteste' com custo 4; no programa, menu 2: senha e hash colado

Fazendo o caminho inverso: gero um hash no site, e o nosso programa o verifica na opção 2. Aparece 'Senha CORRETA'. Se eu mudar uma letra da senha, aparece 'INCORRETA'.

Encerramento (Isabela e Paulo, ~0:15)

Resumindo: o bcrypt usa sal aleatório e custo ajustável para tornar cada palpite de senha

muito mais caro, e a nossa implementação em Python puro é compatível com o bcrypt oficial. Obrigado por assistir!

Dicas de gravação

- Gravem em partes (slides, depois código e terminal) e juntem; é mais fácil de refazer.
- Use custo 4 ou 5 no menu e no site. Custos altos demoram muito em Python puro.
- Use só senhas de teste no site, nunca senhas reais.
- Se o site gerar hash \$2a\$ ou \$2y\$, o programa aceita na verificação.
- Entreguem o vídeo no Classroom junto com o código, até as 19h do dia da apresentação.